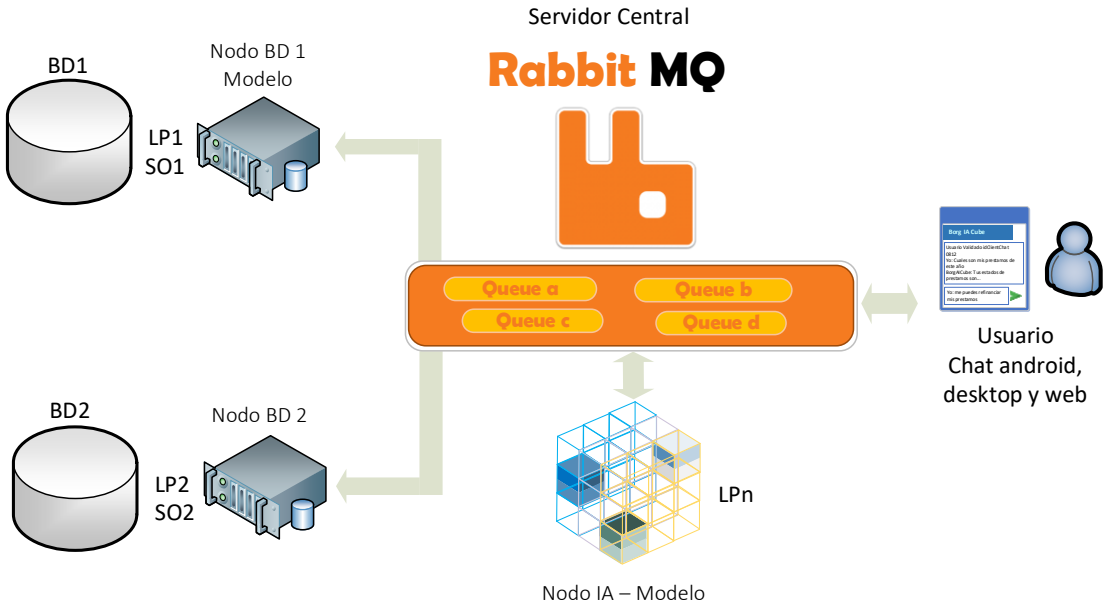
## Escuela de Ciencias de la Computación

## Practica 03 2026-II
## CC4P1 Programación Concurrente y Distribuida

Sistema Distribuido de CNN con Chat Multiplataforma y Middleware RabbitMQ

Usted debe diseñar e implementar un sistema distribuido modular basado en el concepto **Open AI Cubes**, donde cada "cubo" representa una capacidad de IA en una dimensión específica. El sistema procesará **imágenes** enviadas por usuarios a través de un **chat** y devolverá un diagnóstico generado por un modelo de Red Neuronal Convolucional (CNN) entrenados con **datasets propios** seleccionados por el alumno, No usar librería de IA como **TensorFlow, PyTorch, etc**.

La comunicación entre los nodos del sistema debe realizarse **exclusivamente mediante sockets propios implementados con el SDK estándar del lenguaje elegido**, y **RabbitMQ actuará como middleware central** para la mensajería asíncrona entre el backend, los nodos de IA y las bases de datos.




**No se permite** el uso de WebSocket, Socket.IO, ni librerías externas de conexión distintas a RabbitMQ y al SDK base de su lenguaje de programación L.

**Objetivo del examen:** Demostrar dominio de:

- **Concurrencia** (hilos, sincronización, evasión de condiciones de carrera).
- **Distribución** (nodos en máquinas/SO distintos).
- **Comunicación por sockets con RabbitMQ** como middleware.
- **Integración de IA (CNN + MLP)** usando datasets propios para cada modelo (a eleccion de cada grupo).
- **Dos bases de datos heterogéneas** (ej. MariaDB, PostgreSQL, MySQL).
- **Múltiples plataformas cliente**: Android, Desktop y Web.

## Requisitos de IA (Micro Chunks con Datasets Propios)

Cada alumno (o grupo) deberá **seleccionar un modelo para Red Neuronal Convolucional (CNN) para entrenarlo** y **sus respectivos datasets** de su elección, por ejemplo:

| Modelo ejemplos | Dataset sugerido | Tarea |
|---|---|---|
| **Modelo A** | Chest X-Ray Images (Neumonía) | Clasificación binaria: normal vs. neumonía |
| **Modelo B** | HAM10000 (Lesiones cutáneas) | Clasificación multiclase: benigna vs. maligna |

Cada modelo debe entrenarse y testearse en concurrente, paralelo distribuido, y estar compuesto por:

- **Micro chunk de Convoluciones:** Extracción de características (filtros, ReLU, pooling).
- **Micro chunk de MLP:** Clasificador final (capas densas + Softmax).

**Requisito de datasets:** El alumno debe documentar en el informe la fuente del dataset, el preprocesamiento aplicado y las métricas de entrenamiento (como por ejemplo accuracy, loss, matriz de confusión).

**Requisito de BD1:** Su modelo tendrá **su propio base de datos** (motores distintos) donde se almacenen:

- Metadatos de las imágenes procesadas.
- Resultados de la clasificación (etiqueta, probabilidad, fecha).
- Historial de consultas del usuario.
- etc…

**Requisito de BD2:** Su sistema tendrá otra base de datos (motores distintos) donde tenga información que sirva de validación a la BD1, estos se validaran, comunicaran mediante el Middleware Rabbit MQ. Estos pueden incluir información de:

- Datos para la validación del cliente.
- Datos para la validación del producto.
- Datos para la validación de la imagen.
- etc...

## Arquitectura Distribuida Requerida (con RabbitMQ)

El sistema debe integrar los siguientes módulos, **desplegados en nodos separados** (mínimo 3 nodos físicos o máquinas virtuales, con SO distintos):

| Módulo | Función | Comunicación |
|---|---|---|
| Nodo Cliente Android | App móvil (Kotlin/Java) que permite al usuario enviar datos/ imagen por chat y recibir el diagnóstico. | Sockets TCP propios hacia el Backend. |
| Nodo Cliente Desktop | Aplicación de escritorio (GUI de su Lenguaje de programación L) con la misma funcionalidad de chat. | Sockets TCP propios hacia el Backend. |
| Nodo Cliente Web | Interfaz web (HTML/JS + servidor HTTP propio con sockets) que consume el servicio. | Sockets TCP propios hacia el Backend (no WebSocket). |
| Nodo Backend (Servidor Central) | Recibe las consultas de los 3 clientes, valida usuarios, enruta los datos/imágenes a RabbitMQ y devuelve la respuesta. | Sockets TCP con clientes + RabbitMQ con nodos IA/BD. |
| Nodo IA – Modelo | Ejecuta la CNN (con su dataset) sobre las datos/imágenes recibidas. | Consume/publica en RabbitMQ. |
| Nodo BD 1 Modelo | Gestiona datos del cliente con el modelo CCN A (ej. PostgreSQL/MongoDB). | Publica/consume en RabbitMQ. |
| Nodo BD 2 | Gestiona datos con los demás nodos (ej. MariaDB/MySQL). | Publica/consume en RabbitMQ. |

Nota: El Frontend Web debe ser servido por un servidor HTTP propio implementado con sockets (no se permite Flask/Django/Spring Boot como servidor web; sí se permite usar el SDK de sockets para servir HTML/JS).

RabbitMQ debe configurarse con colas dedicadas: cola_modeloA, cola_modeloB, cola_resultadosA, cola_resultadosB, y colas de control (heartbeat, logs).

Protocolo de Comunicación entre Nodos

- Entre Clientes y Backend (Sockets TCP propios): Protocolo de mensajes definido por el alumno (JSON serializado manualmente o texto delimitado). Debe incluir por ejemplo: idCliente, tipoConsulta (imagen), payload (bytes de la imagen codificada en Base64), y timestamp.
- Entre Backend y Nodos IA/BD (RabbitMQ): Mensajes publicados en las colas correspondientes, con formato definido por el alumno (JSON o texto plano). Debe incluir por ejemplo idSolicitud, imagen, modeloDestino, timestamp.
- Prohibido: WebSocket, Socket.IO, middleware distinto a RabbitMQ, librerías de terceros para conexión.

**Diagrama de protocolo obligatorio** en el informe: mostrar el flujo Cliente → Backend → RabbitMQ → Nodo IA → RabbitMQ → Nodo BD → RabbitMQ → Backend → Cliente.

## Bases de Datos Heterogéneas (Obligatorio)

El sistema debe usar **exactamente 2 bases de datos de motores distintos** (ej. MariaDB y PostgreSQL, o MySQL y MariaDB, o PostgreSQL y MySQL). Cada una debe estar asociada a un modelo de IA y desplegada en un nodo independiente.

**Requisito:** Los scripts de creación de ambas BD deben entregarse. Las escrituras concurrentes deben manejarse con hilos y mecanismos de sincronización (transacciones, locks, etc.) para evitar corrupción de registros.

Se tienen 3 o más nodos con:

Sistema Operativos, SO1, SO2 y SO3, donde SO1 <> SO2 <> SO3

Bases de Datos, BD1, BD2 y BD3, donde BD1 <> BD2 <> BD3

Lenguajes de Programación, LP1, LP2 y LP3, donde LP1<> LP2 <> LP3

Donde cada uno de los LP1, LP2, LP3 se encuentran en un nodo independiente.

El Middleware se encuentra en un nodo.

Desarrollando el servidor en internet o en la nube (opcional).

Graficar la arquitectura diseñada.

Graficar el diagrama de protocolo.

No usar websocket, socketio.

No usar librerías de terceros, solamente usar el sdk original del lenguaje de programación original.

Explicar el desarrollo del programa puntualmente.

Desplegar el programa en redes.

Validar las respectivas BD1, BD2 y BD3 los cambios

Evaluar el desempeño haciendo un script donde el cliente ingrese aleatoriamente 1000 nuevos registros y ver métricas de evaluación del sistema distribuido.

Poder ingresar en el cliente registros uno por uno.

Como base realizar el código en terminal. En algunos casos dependiendo el “SOn” podrán realizar Interfaz Gráfica.

Usar hilos para mejorar el desempeño e evitar corrupción de registros.

## Pruebas de Rendimiento y Saturación

El alumno debe realizar las siguientes evaluaciones y presentar **gráficas comparativas**:

1. **Secuencial:** 1 proceso, 1 hilo.

2. **Paralelo:** 1 máquina con N núcleos físicos, H hilos por núcleo.
3. **Distribuido:** M máquinas, 1 núcleo, 1 hilo por núcleo.
4. **Paralelo Distribuido:** M máquinas con N núcleos, H hilos por núcleo.

**Prueba de saturación:** Los clientes (Android, Desktop y Web) deben enviar **1000 transacciones** (datos/imágenes) con un **delay aleatorio** entre envíos para saturar el sistema. Se deben medir:

- Tiempo de respuesta promedio.
- Throughput (transacciones/segundo).
- Uso de CPU y memoria en cada nodo.
- Tasa de errores.
- Comportamiento de las colas de RabbitMQ (profundidad, mensajes pendientes).

**Gráficas requeridas:** Latencia vs. número de nodos, Throughput vs. configuración (secuencial/paralelo/distribuido), y uso de RabbitMQ bajo carga.

Subir solamente en UNIVIRTUAL en su tiempo.

- Comprimido consta
    - Subir solo los códigos fuente de extensión en el LP1, LP2, LP3, LPn, LPn+1....
    - Scripts de la BD1, BD2 y BD3.
    - Si suben en otro tipo de extensión que no se solicitó se amonestara con puntos a excepción del dataset y los comando sql.
    - PDF Informe.
    - PDF Presentación.